HR Tech · Developer Tools · AI-Powered Assessment

StackScore

Hire for how engineers actually work.

Team: **StackScore** Size: **Solo** Contact: **greyinofficial@gmail.com** Stack: Next.js · Gemini · Claude · GCP

PROBLEM

Technical hiring measures what candidates memorize — not how they engineer. LeetCode scores don't capture requirements clarity, system design judgment, or adaptability under real constraints. Every panel interviews differently and results aren't comparable. Senior engineers spend hours in live reviews that produce no structured, reusable signal.

PROBLEM STATEMENT

Technical hiring measures what candidates memorize — not how they engineer.

01

Wrong Signal

LeetCode-style tests measure algorithmic recall and practice-driven pattern matching. They say nothing about system design judgment, requirements clarity, or adaptability under pressure — the skills that matter in a real senior role.

A candidate who aces dynamic programming may not design a URL shortener that survives a 10× traffic spike.

02

No Standard Rubric

Every panel interviews differently. Results are undocumented, uncomparable across roles or companies, and irreproducible. The same candidate passes or fails based on who happened to be available — not structured capability.

Hire decisions reduce to interviewer intuition, not documented engineering capability.

03

High Cost, Low Output

A live system-design interview costs 2–3 senior engineering hours per candidate. No feedback is given to candidates. HR can't track scores over time. There's no data layer for improving the process.

Companies spend hours per candidate and get a binary yes/no with no reusable signal.

PROPOSED SOLUTION

A 5-stage AI simulation of real engineering work

Stage 1 · 5 min

Clarify

Candidate Q&A with AI stakeholder to scope a deliberately ambiguous brief

Stage 2 · 10 min

Propose

Written system design against the requirements uncovered in Stage 1

Stage 3 · 20 min

Build

Code the core module; runs in a sandboxed Node.js executor

Stage 4 · 10 min

Adapt

AI injects a mid-task constraint twist; candidate updates design & code

Stage 5 · 15 min

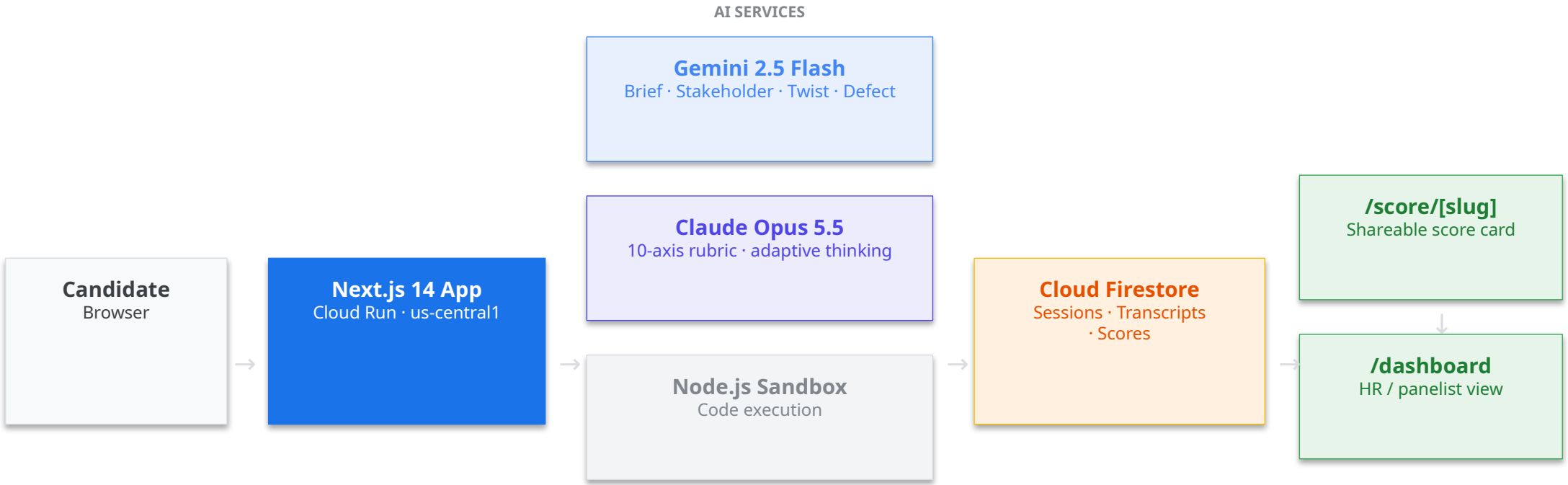
Debug

Candidate finds and fixes a subtle defect AI seeded into their own code

25 scenarios from a 5×5 matrix: problem domains × engineering constraint pillars · **Gemini 2.5 Flash** generates · **Claude Opus 5.5** judges (adaptive thinking) · Output: **StackScore 0–100** · role-calibrated · shareable URL

SOLUTION ARCHITECTURE

Dual-AI pipeline on Google Cloud



GOOGLE AI TOOLS

Built end-to-end on Google Cloud

🧪 Google AI Studio

Gemini API key provisioned via AI Studio. Used in production for all generation calls — no Vertex AI billing complexity needed for the MVP.

aistudio.google.com

📦 @google/genai SDK

Official JS SDK for Gemini API. genai.models.generateContent() powers brief generation, stakeholder voice, twist injection, and defect seeding.

model: "gemini-2.5-flash"

☁️ Cloud Run

Serverless container hosting for the Next.js 14 app. Scales to zero between sessions; 512 MiB RAM. Deployed automatically via Cloud Build CI/CD pipeline.

us-central1 · --allow-unauthenticated

🔥 Cloud Firestore

Document database storing sessions, stage transcripts, and score cards. Queried in the HR dashboard for pass/fail threshold filtering by role.

collections: sessions · transcripts · scores

🏗️ Cloud Build

CI/CD pipeline via cloudbuild.yaml. Builds Docker image, pushes with COMMIT_SHA tag, and deploys to Cloud Run in one gcloud builds submit command.

cloudbuild.yaml · Docker · Cloud Run deploy

🔑 Secret Manager

Stores GEMINI_API_KEY, ANTHROPIC_API_KEY, FIREBASE_PRIVATE_KEY. Injected into Cloud Run at deploy time via --set-secrets. Zero secrets in source code.

--set-secrets GEMINI_API_KEY=...latest

GOOGLE AI MODELS

Generate with Gemini · Judge with Claude · Fallback with Haiku

Gemini 2.5 Flash

GENERATION

via @google/genai · API key mode

→ Problem brief generation — deliberately underspecified, stakeholder-framed, with hidden constraint woven in implicitly

→ Stakeholder voice simulation — answers clarifying questions in character without revealing the constraint category

→ Mid-task constraint twist injection — a natural 'scope change from leadership' reframing the session's pressure

→ Debug defect seeding — plants one non-trivial logic error (off-by-one, missing edge case) in the candidate's own code

```
genai.models.generateContent({ model: "gemini-2.5-flash", contents: prompt })
```

Claude Opus 5.5

JUDGING

via Anthropic SDK · adaptive thinking

→ Rubric-constrained 10-axis evaluation — each axis scored 0–100 independently to prevent holistic-impression bleed

→ thinking: { type: "adaptive" } lets the model reason through nuanced design tradeoffs before committing to scores

→ Role-calibrated prompt — Junior/Mid/Senior expectation bars stated explicitly so the same rubric spans all levels

→ Returns strict JSON: axis scores + 1–2 sentence feedback per axis, persisted to Firestore and displayed on score card

```
thinking: { type: "adaptive" }, model: "claude-opus-5-5", max_tokens: 4096
```

Claude Haiku 5.5

FALLBACK

fallback generation only

→ Activated when Gemini API is unavailable or returns empty — seamless in-session fallback

→ Runs the same generation prompts so the assessment never fails mid-session for the candidate

→ Low-latency responses even under fallback conditions

WHAT'S BUILT · ROADMAP

MVP delivered in 7 days — roadmap for scale

✅ Built — MVP (7 days)

- ✅ 5-stage assessment workflow (Clarify → Propose → Build → Adapt → Debug)
- ✅ 25-scenario matrix — 5 problem domains × 5 engineering constraint pillars
- ✅ Dual-AI pipeline: Gemini 2.5 Flash (generation) + Claude Opus 5.5 (judging)
- ✅ 10-axis rubric, weighted StackScore 0–100 with per-axis written feedback
- ✅ Role calibration — Junior / Mid / Senior adjusts judge prompt expectations
- ✅ Per-stage countdown timers with 1-minute expiry warning
- ✅ Fresher hints mode — togglable per-stage tips for junior candidates
- ✅ Shareable score card URL (/score/[slug]) with axis breakdown & feedback
- ✅ HR/panelist dashboard (/dashboard) with pass/fail threshold filtering
- ✅ Cloud Run + Cloud Build CI/CD + Secret Manager deployment pipeline
- ✅ 71-test Playwright suite — 100% passing across all features and stages

⟶ Roadmap — Post-MVP

- ⟶ ATS/HRIS connectors — Greenhouse, Lever, Workday integration for automated candidate invite & score sync
- ⟶ Multi-language code execution — Python, Go, Java alongside existing Node.js sandbox
- ⟶ Benchmark database — percentile rankings by role level across anonymized sessions
- ⟶ Dynamic rubric reweighting — K4 (Security) auto-upweights when session constraint is C4
- ⟶ Voice-based Clarify stage — speech-to-text for a more natural stakeholder Q&A
- ⟶ Custom rubric builder — companies configure axis weights for their specific hiring bar
- ⟶ Panelist scheduling + candidate invite flow — HR sends link, candidate takes assessment async
- ⟶ Team interview mode — pair programming assessment variant for collaborative roles

Thank you

StackScore

Hire for how engineers actually work.

greyinofficial@gmail.com

Startup Buildathon India 2026 · Google DeepMind × Google Cloud